

Events 3

Brainster Web Development Academy



Table of contents

01

The “event” object

02

e.target & e.currentTarget

03

Form events and e.preventDefault()

04

Javascript Timers

05

Delayed execution

06

Repetitive execution

07

Clearing intervals



The “event” object

- There are many types of events, but all of those are using the default Event object interface.
- Event itself contains the **properties** and **methods** which are common to all events.

```
let myBtn = document.querySelector('button');  
  
function onClick(event) { console.log(event); }  
  
myBtn.addEventListener('click', onClick);
```



e.Target & e.currentTarget

- We know that we can get data and manipulate a DOM element if we have assigned it to a variable or if we know a way of selecting it (class, id, etc.).
- However, when handling events, we don't really have to know the element the handler was assigned to.
- This is especially practical if we happen to work with multiple elements - it isn't really practical to create a new variable for each of them.

7

```
<ul class="todo-list">  
  <li class="item">Walk your dog</li>  
</ul>
```

```
const list = document.querySelector(".todo-list");  
  
list.addEventListener("click", e => {  
  console.log(e.target);  
  // Output: <li class="item">Walk your dog</li>  
  console.log(e.currentTarget);  
  // Output: <ul class="todo-list"></ul>  
});
```



The “event” object

```
function onClick(event) {  
    const buttonClicked = event.target;  
    console.log(buttonClicked.text, 'clicked');  
}
```

7

```
const buttons = document.querySelectorAll('button');  
  
for (let i = 0; i < buttons.length; i++) {  
    buttons[i].addEventListener('click', onClick);  
}
```



Exercise 1

1. Write a document with 10 different buttons (numbered from 1-10)
2. Add an event handler to all of them such as:
 - a. When the button gets clicked, it logs to the console the number it carries

For example:

7

If I have a button with “5” on it, when it gets clicked, it logs “5” to the console.



Exercise 2

1. Make two inputs, one for username, one for password.
2. Make reusable functions **onFocus** and **onBlur**
 - a. onFocus function should put border on the currently selected element
 - b. onBlur function should remove the border on the element that it was fired on
3. Add the functions to both of the inputs



Form events & e.preventDefault()

- Form tags also come with their own events - most notably, the 'submit' event.
- **'submit'** is special in the sense that it triggers any time we submit a form - when pressing enter, when pressing a submit button within a form etc. It's just a convenient way of catching all submission types.
- Additionally, it gives us a way of stopping the page from reloading (the default behavior when we submit a form).

```
const myForm = document.querySelector('form');
```

```
function onSubmit(event) {  
    event.preventDefault();  
    console.log('Form was submitted');  
}
```

```
myForm.addEventListener('submit', onSubmit);
```



Form events & e.preventDefault()

- The other useful form event is “**reset**”. If we give a button within a form the type “reset”, it will simply reset the form to it's default state.
- Much alike the “submit” event, we can subscribe to the “reset” event and be able to change it's behavior.



Exercise 3

1. Make a form that represents adding an address
 - a. It should consist of inputs for: address1, address2, zip code, city, state and country
2. Add two buttons, submit and reset
3. On click on the submit button, the address should be shown in an alert box
4. On click on the reset button, the form should reset to its initial state



Aside: links and click

- As we know, any link with an href attribute leads to a different page, or to an internal (hash) link in the document - that is it's **default behaviour**.
- An "a" element can have a click handler, with one important caveat: we want to also use **.preventDefault()** on the event handler if we'd like to prevent the link from going to a different page.

7

```
const link = document.querySelector('a') // <a href="google.com">go</a>
```

```
function handler(e) {  
  e.preventDefault();  
  console.log('handling a click from a without a page change');  
}
```

```
link.addEventListener('click', handler);
```



Javascript Timers

Brainster Web Development Academy



Delayed execution

- If we ever need to trigger something **once** after a certain time, we can use the **setTimeout()** method.

```
function logSomething() {  
    console.log('something');  
}
```

```
setTimeout(logSomething, 1000) // will execute after 1 second
```



Repetitive execution

- If we ever need to trigger something **non-stop**, we can use the **setInterval()** method.

```
function logSomething() {  
    console.log('something');  
}
```

```
setInterval(logSomething, 1000) // will execute every 1 second
```



Clearing intervals

- To stop an interval from execution, we use the **clearInterval()** method.

Note: to clear the interval, you must save it (its id) in a variable first.

```
const myBtn = document.querySelector('button');  
const myInterval = setInterval(logSomething, 1000);
```

```
myBtn.addEventListener('click', function() {  
    clearInterval(myInterval);  
});
```



Exercise 4

1. Create two functions, ping and pong.
2. Both should simply log “ping” or “pong”.
3. Inside the ping function make a timer using `setTimeout` which will invoke the pong function after 1000 milliseconds.
4. Create one interval outside everything which will invoke the ping function every 2000 milliseconds.
5. Create a button which on click stops the ping-pong.



Homework

Create a simple game: guessing a number the computer generates.

1. Create an input element and a button next to it (in html)
2. Create a function called game which:
 - a. Generates a random number from 1-50
 - b. Gets the value of the input element
 - c. Compares whether the two values are the same
 - d. If they are, alerts "You win"
 - e. If not, it alerts "Try again"
3. 3. Attach an event listener to the button that invokes the function when clicked

